

# Label-Private Split Federated Learning Framework for MIMIC-IV Mortality Prediction

## 1. Framework Summary

This report presents the implemented label-private Split Federated Learning framework for distributed healthcare AI using non-IID MIMIC-IV client partitions. The framework was designed to move beyond a notebook-level simulation and establish a real client-server training system in which local clients communicate with a remote server model through an API.

The central objective is to support collaborative model training while preserving the privacy boundary of clinical data. In the implemented system, raw patient records and mortality labels remain local to each client. The server receives only intermediate activations and gradient signals required for split learning.

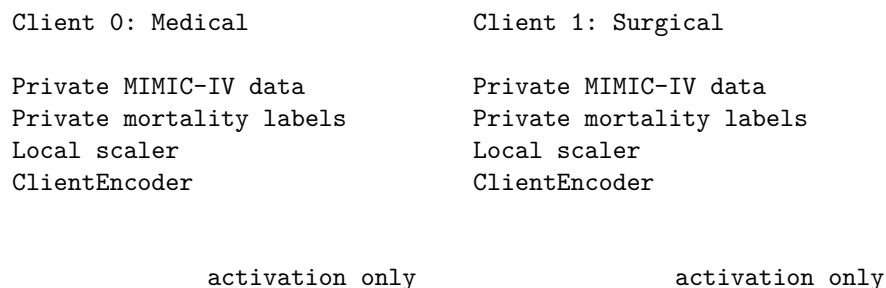
The framework has been validated using a real distributed setup:

| System Component    | Runtime Location                          |
|---------------------|-------------------------------------------|
| Client processes    | Local machine                             |
| Server model        | Google Colab                              |
| Communication layer | FastAPI exposed through Cloudflare tunnel |
| Learning protocol   | Label-private Split Federated Learning    |
| Aggregation method  | FedAvg over client encoders               |

## 2. Architectural Overview

The framework follows a split client-server architecture. Each client owns its private data, labels, scaler, and encoder. The server owns only the server-side prediction model and aggregation logic.

### Label-Private SplitFed Architecture



Remote SplitFed Server: Google Colab

FastAPI communication layer  
ServerTopModel  
Temporary activation-gradient context  
FedAvg aggregation state

activation only

Client 2: Cardiac

Private MIMIC-IV data  
Private mortality labels  
Local scaler  
ClientEncoder

Forward learning signal:

Clients    activations   > Server    logits   > Clients

Backward learning signal:

Clients    logit gradients   > Server    activation gradients   > Clients

Federated aggregation:

Client encoders    weights   > Server FedAvg    global encoder   > Clients

Privacy boundary:

|                     |                                                  |
|---------------------|--------------------------------------------------|
| Raw patient records | stay on local clients                            |
| Mortality labels    | stay on local clients                            |
| Server receives     | activations, logits gradients, encoder weights   |
| Server returns      | logits, activation gradients, aggregated encoder |

### 3. Research Context

Healthcare AI systems often require collaborative learning across multiple clinical sources, such as hospital units, wards, institutions, or device groups. Direct centralization of patient data is difficult because of privacy, governance, and regulatory constraints.

Federated Learning reduces the need for centralized data by training models across distributed clients. Split Learning separates a neural network into client-side and server-side components, allowing clients to send intermediate activations rather than raw features. Split Federated Learning combines these two ideas:

- split model execution between client and server,
- distributed client-side training,
- periodic aggregation of client-side model parameters.

The implemented framework applies this idea to MIMIC-IV mortality prediction using three non-IID clinical partitions.

#### 4. Dataset Partitioning

The dataset is divided into three non-IID clients based on clinical partition type:

| Client ID | Partition | Role In Framework                |
|-----------|-----------|----------------------------------|
| Client 0  | Medical   | Local medical patient partition  |
| Client 1  | Surgical  | Local surgical patient partition |
| Client 2  | Cardiac   | Local cardiac patient partition  |

Each client has its own local data distribution. This is important because real healthcare systems rarely have identical patient populations across institutions or clinical units.

Each client locally stores:

- private MIMIC-IV CSV partition,
- mortality labels,
- preprocessing scaler,
- pretrained client encoder,
- local train/validation dataloaders.

The server does not load or store the raw MIMIC-IV client files.

#### 5. System Components

The implemented framework consists of four major components:

| Component            | Description                                                      |
|----------------------|------------------------------------------------------------------|
| Client data pipeline | Loads and preprocesses local client data using the saved scaler  |
| ClientEncoder        | Local neural encoder that converts ICU features into activations |
| ServerTopModel       | Remote neural prediction head hosted on the server               |
| FedAvg module        | Aggregates client encoder parameters after each training round   |

The codebase separates these into client-side, server-side, and shared modules:

```

src/sfl/client/
  dataset.py      local data loading and preprocessing
  api.py          client-side API communication
  trainer.py      label-private SplitFed training logic
  multi_runner.py multi-client experiment runner

src/sfl/server/
  app.py          FastAPI application and endpoints
  trainer.py      server-side forward/backward logic
  state.py        server state, metrics, checkpointing
  fedavg.py       encoder aggregation

src/sfl/common/
  models.py       ClientEncoder and ServerTopModel
  metrics.py      AUROC, AUPRC, F1
  serialization.py tensor and model-state serialization

```

## 6. Model Architecture

The neural network is split into a client-side encoder and a server-side prediction head.

### 6.1 Client-Side Encoder

The client encoder receives the local ICU feature vector and produces a compact activation representation.

```

Input: 266 ICU clinical features
Linear(266 -> 64)
BatchNorm1d(64)
ReLU
Dropout(0.5)
Linear(64 -> 32)
BatchNorm1d(32)
ReLU
Output: 32-dimensional smashed activation

```

The activation vector is the only learned feature representation transmitted from the client to the server during training.

### 6.2 Server-Side Model

The server-side model receives the 32-dimensional activation and produces a mortality prediction logit.

```

Input: 32-dimensional activation
Linear(32 -> 64)
BatchNorm1d(64)

```

ReLU  
Dropout(0.3)  
Linear(64 -> 1)  
Output: mortality logit

### 6.3 Parameter Counts

| Model Part     | Parameter Count |
|----------------|-----------------|
| ClientEncoder  | 19,360          |
| ServerTopModel | 2,305           |

This design keeps the server-side model lightweight while allowing each client to locally transform private patient features before communication.

## 7. Label-Private SplitFed Training Flow

The implemented protocol is label-private. This means that mortality labels are never sent to the server.

The training flow for one batch is:

- Step 1: Client loads local feature batch  $X$  and private label batch  $y$ .
- Step 2: Client computes activation  $A = \text{ClientEncoder}(X)$ .
- Step 3: Client sends activation  $A$  to the server.
- Step 4: Server computes logit  $z = \text{ServerTopModel}(A)$ .
- Step 5: Server returns  $z$  to the client.
- Step 6: Client computes  $\text{BCEWithLogitsLoss}(z, y)$  locally.
- Step 7: Client computes gradient  $dL/dz$ .
- Step 8: Client sends  $dL/dz$  to the server.
- Step 9: Server backpropagates through  $\text{ServerTopModel}$ .
- Step 10: Server returns  $dL/dA$  to the client.
- Step 11: Client backpropagates through  $\text{ClientEncoder}$  using  $dL/dA$ .
- Step 12: Client updates local encoder parameters.

This flow preserves the core privacy boundary:

| Item                  | Location                        |
|-----------------------|---------------------------------|
| Raw clinical features | Client only                     |
| Mortality labels      | Client only                     |
| Client encoder        | Client side                     |
| Server model          | Server side                     |
| Transmitted to server | Activations and logit gradients |
| Returned to client    | Logits and activation gradients |

## 8. Federated Aggregation

After each training round, the framework performs FedAvg over the client encoders.

The aggregation flow is:

1. Each client trains locally through split learning for one round.
2. Each client submits its encoder weights to the server.
3. The server waits for all three client submissions.
4. The server computes weighted average encoder parameters.
5. The global encoder state is sent back to all clients.
6. Each client replaces its local encoder with the aggregated global encoder.

The server uses client training-set size as the aggregation weight. This allows clients with more samples to contribute proportionally to the global encoder update.

FedAvg is applied only to the client-side encoders. The server-side model is trained directly through the activation-gradient flow.

## 9. API-Based Client-Server Connection

The real client-server connection is implemented using FastAPI. The server runs as an API service, and each local client sends HTTP requests to the server.

During local testing, the server can run on the same machine. During distributed testing, the server runs on Google Colab and is exposed using a Cloudflare public tunnel.

The main API endpoints are:

| Endpoint                    | Purpose                                                      |
|-----------------------------|--------------------------------------------------------------|
| GET /health                 | Confirms that the server is running                          |
| POST /register_client       | Registers client metadata such as client ID and sample count |
| POST /option-b/forward      | Receives client activations and returns server logits        |
| POST /option-b/backward     | Receives logit gradients and returns activation gradients    |
| POST /predict               | Produces validation logits from activations                  |
| POST /fedavg/submit_encoder | Receives client encoder weights for aggregation              |
| GET /fedavg/global_encoder  | Returns the aggregated global encoder                        |
| POST /checkpoint/server     | Saves the server-side model checkpoint                       |

The public URL itself does not show a dashboard. Opening the base URL may return:

```
{"detail": "Not Found"}
```

This is expected because the server is an API service, not a web interface. The correct health-check endpoint is:

```
/health
```

## 10. Server-Side State Management

The server maintains temporary state required for split learning:

- registered client metadata,
- active forward contexts for label-private training,
- server model parameters,
- optimizer state,
- FedAvg encoder submissions,
- latest global encoder,
- server-side metrics,
- optional server checkpoints.

For label-private training, the server temporarily stores the activation tensor and associated logits during the forward call. When the client sends back the logit gradient, the server uses this context to compute the activation gradient. The context is removed after the backward step.

This ensures each forward-backward pair remains linked while avoiding long-term storage of client activations.

## 11. Validation Flow

Validation is also privacy-preserving with respect to labels.

The validation process is:

1. Client computes activation for validation samples.
2. Client sends activation to the server prediction endpoint.
3. Server returns logits.
4. Client computes AUROC, AUPRC, and F1 locally using private labels.

The server does not receive validation labels.

## 12. Experimental Setup

A medium-scale experiment was completed using the real client-server framework.

| Item             | Value                            |
|------------------|----------------------------------|
| Clients          | 3                                |
| Training rounds  | 5                                |
| Training batches | 200 batches per client per round |
| Validation       | Limited validation batches       |
| Batch size       | 64                               |
| Client runtime   | Local machine                    |
| Server runtime   | Google Colab                     |
| Communication    | Cloudflare tunnel                |
| Aggregation      | FedAvg after each round          |
| Protocol         | Label-private SplitFed           |

All major request types completed successfully during testing, including forward, backward, prediction, FedAvg submission, and global encoder retrieval.

## 13. Current Experimental Metrics

### 13.1 AUROC Across Rounds

| Round | Medical AUROC | Surgical AUROC | Cardiac AUROC | Mean AUROC |
|-------|---------------|----------------|---------------|------------|
| 1     | 0.9515        | 0.9636         | 0.9749        | 0.9633     |
| 2     | 0.9448        | 0.9670         | 0.9749        | 0.9622     |
| 3     | 0.9449        | 0.9666         | 0.9775        | 0.9630     |
| 4     | 0.9467        | 0.9662         | 0.9774        | 0.9634     |
| 5     | 0.9476        | 0.9677         | 0.9781        | 0.9644     |

### 13.2 Round 5 Detailed Metrics

| Client   | Loss   | AUROC  | AUPRC  | F1     |
|----------|--------|--------|--------|--------|
| Medical  | 0.5507 | 0.9476 | 0.8297 | 0.6838 |
| Surgical | 0.3903 | 0.9677 | 0.8543 | 0.7432 |
| Cardiac  | 0.3147 | 0.9781 | 0.8363 | 0.6859 |

### 13.3 Training Stability

The medium-scale experiment showed stable multi-client behavior:

- all three clients completed each round,
- FedAvg completed after every round,
- client losses decreased over training,
- validation AUROC remained high across all clients,
- the mean AUROC reached 0.9644 by round 5.

## 14. Communication Cost

The communication estimate below covers the core activation-gradient exchange in the medium-scale run.

Per client per round:

| Communication Item                      | Bytes     |
|-----------------------------------------|-----------|
| Activations sent to server              | 1,638,400 |
| Activation gradients returned to client | 1,638,400 |
| Total per client per round              | 3,276,800 |

For all three clients:

| Communication Item  | Bytes      |
|---------------------|------------|
| Total per round     | 9,830,400  |
| Total over 5 rounds | 49,152,000 |

This estimate covers the main activation and activation-gradient tensors. It does not include smaller metadata, validation prediction requests, or encoder-weight transfer during FedAvg.

## 15. Privacy Characteristics

The implemented framework provides the following privacy properties:

| Privacy Aspect         | Framework Behavior                                          |
|------------------------|-------------------------------------------------------------|
| Raw feature privacy    | Raw MIMIC-IV records remain local                           |
| Label privacy          | Mortality labels remain local                               |
| Server-side visibility | Server sees activations, logits, and gradients              |
| Data centralization    | No centralized patient dataset is used by the server        |
| Client independence    | Each client owns its local preprocessing and encoder update |

The framework does not claim that activations are cryptographically secure. Instead, this phase establishes the label-private SplitFed training substrate. Additional security mechanisms such as activation perturbation, differential privacy, secure aggregation, or quantum-inspired feature scrambling can be layered on top of this substrate in later research extensions.

## 16. Implemented Outputs

The framework generates the following artifacts:

| Artifact                   | Description                                     |
|----------------------------|-------------------------------------------------|
| Client encoder checkpoints | Updated encoder weights after SplitFed training |
| Server checkpoint          | Server-side model parameters                    |
| Client metrics JSONL       | Per-client training and validation events       |
| Experiment summary CSV     | Round-wise metrics for analysis                 |
| Experiment summary JSONL   | Structured event log                            |
| Server metrics JSONL       | Server-side request and training events         |

These outputs support later table generation, plotting, and paper result analysis.

## 17. Summary

The implemented system establishes a functioning label-private Split Federated Learning framework for MIMIC-IV mortality prediction. It connects local clients and a remote Colab server through a real API-based workflow, performs activation-gradient exchange, keeps labels local, updates client encoders through returned activation gradients, and aggregates client models through FedAvg.

The medium-scale experiment validates that the system is operational across three non-IID clinical clients and achieves stable AUROC performance. The framework therefore provides the core technical foundation for privacy-preserving collaborative healthcare AI experiments in this research project.